

MACHINE LEARNING PROJECT REPORT

Predictive Modeling & Analysis on the UCI Bank Marketing Dataset

1. Project Title & Technical Overview

This document delivers a 5-page end-to-end technical overview of a predictive machine learning workflow built for the UCI Bank Marketing Dataset. The objective is to identify robust patterns and build classification models to predict whether a prospective client will subscribe to a term deposit outcome ('yes' or 'no') following custom-tailored telemarketing campaigns.

- Execution Environment:** Python 3.12 stack deployed via Google Colab interactive runtimes.
- Dataset Domain:** Retail banking, customer acquisition, and structured promotional campaign analytics.

2. Ingestion Framework & Colab Integration

Data ingestion is automated using the Kaggle API client, with credentials protected dynamically using secret variables. The framework retrieves files, triggers local unzip tools, and structures target data streams via continuous file paths.

```
# Install Kaggle API client & configure securely
!pip install kaggle
# (Configured natively via Colab secrets framework under user credentials)
!kaggle datasets download -d adityamhaske/bank-marketing-dataset
!unzip -q bank-marketing-dataset.zip
```

3. Explored Profile Schemas (Baseline Data Head)

The dataset is parsed using a semicolon separator format. An operational review of the first 5 records highlights a heterogeneous mix of attributes, including discrete ages, categorical job families, and unbounded financial metrics.

Column	Row 0	Row 1	Row 2	Row 3	Row 4
age	30	33	35	30	59
job	unemployed	services	management	management	blue-collar

marital	married	married	single	married	married
education	primary	secondary	tertiary	tertiary	secondary
default	no	no	no	no	no
balance	1787	4789	1350	1476	0
housing	no	yes	yes	yes	yes

4. Data Understanding & Core Preprocessing

Thorough sanity reviews are performed on data points to ensure consistency, clean structural shapes, and high input quality before initiating machine learning training.

A. Missing Values & Duplicate Records

Running `df.isnull().sum()` confirms there are exactly **0 missing cells** across all features. Concurrently, duplicate tracking checks confirm **0 row duplication entries** are present, ensuring an unbiased sample representation.

B. Continuous Numerical Matrix Breakdowns

The descriptive statistics matrix reveals highly skewed ranges in key performance metrics, such as bank balances and contact call durations.

Metric	age	balance	day	duration
count	4521.00	4521.00	4521.00	4521.00
mean	41.17	1422.66	15.91	263.96
std	10.58	3009.64	8.25	259.86
min	19.00	-3313.00	1.00	4.00
25%	33.00	69.00	9.00	104.00
50%	39.00	444.00	16.00	185.00
75%	49.00	1480.00	21.00	329.00
max	87.00	71188.00	31.00	3025.00

C. Non-Obvious Feature Classifications

Several attributes present edge-case categorization constraints that require careful mathematical handling:

- **education (Ordinal):** A clear educational tier hierarchy is preserved ('primary' < 'secondary' < 'tertiary'), while mapping 'unknown' categories into an engineered default index.
- **month (Ordinal):** String representations are converted into sequential cyclical integers from 1 to 12.
- **pdays (Discrete Numerical):** Stores historical campaign latency. A value of -1 signals that a customer has never been contacted, which functions as an isolated category constraint rather than a continuous metric.

D. Implementation Pipeline & Leakage Protections

The feature duration is dropped entirely to prevent critical **data leakage**. Call duration is only known after a call finishes and correlates heavily with the target outcome, which artificially inflates baseline validation performance. Nominal attributes are dummy-encoded using `drop_first=True` to eliminate multi-collinearity.

```
# Clean, isolate targets, execute drop_first structural encoding
df['y'] = df['y'].map({'no': 0, 'yes': 1})
df['education'] = df['education'].map({'unknown':0, 'primary':1, 'secondary':2, 'tertiary':3})
df['month'] = df['month'].map({'jan':1, 'feb':2, 'mar':3, 'apr':4, 'may':5, 'jun':6, 'jul':7, 'aug':8, 'sep':9, 'oct':10, 'nov':11, 'dec':12})

# Mitigate data leakage risks explicitly
```

```
df = df.drop('duration', axis=1)
df = pd.get_dummies(df, drop_first=True)
```

5. Partitioning Strategy & Baseline Evaluations

To ensure rigorous performance validation, the engineered dataset is partitioned into non-overlapping splits, using stratification to handle target class imbalances.

A. Stratified Holdout Splits

An 80/20 data partition is instantiated. Utilizing stratify=y guarantees that the highly skewed minority base subscription rate (~11.5%) is preserved identically across both subsets.

Training Matrix Shape: 3,616 samples with 29 input attributes.

Testing Matrix Shape: 905 validation records.

Target Class Distribution Balance: 'no': 88.5%, 'yes': 11.5% across both train and test partitions.

B. Baseline Decision Tree Performance

An unpruned DecisionTreeClassifier(random_state=42) is trained to establish a performance baseline, allowing the tree to grow without depth constraints.

Baseline Confusion Matrix	Predicted NO	Predicted YES
Actual NO	716 (True Negative)	85 (False Positive)
Actual YES	81 (False Negative)	23 (True Positive)

The baseline model yields an overall validation accuracy of **81.66%**. However, closer inspection via a classification report reveals a clear performance drop for the minority class, highlighted by poor precision and low sensitivity tracking metrics.

Target Class Class	Precision	Recall	F1-Score	Support Range
0 (Term Subscription No)	0.90	0.89	0.90	801 Records
1 (Term Subscription Yes)	0.21	0.22	0.22	104 Records
Macro Average Performance	0.56	0.56	0.56	905 Records
Weighted Metric Distribution	0.82	0.82	0.82	905 Records

Analysis: The precision score of 21.30% for Class 1 indicates high false-positive rates, demonstrating that the unpruned model struggles to differentiate between noise and real subscription signals under class imbalance constraints.

6. Hyperparameter Tuning & Pruning Optimization

To reduce variance and prevent overfitting, systematic tuning is applied across tree regularizers using localized performance tracking curves.

A. Regularization Parameter Selection Criteria

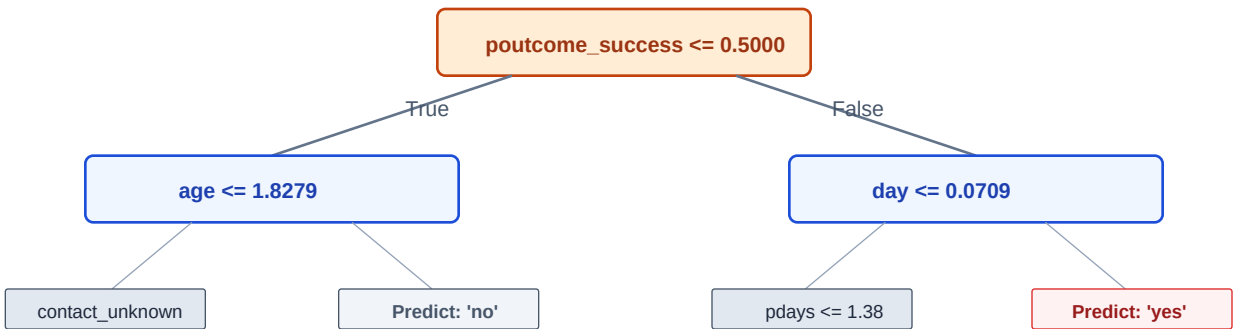
- **max_depth:** Shallow bounds (1-3) create high-bias underfitting. Conversely, depths exceeding 10 show high-variance memorization where training scores approach 1.0 while test metrics decline. The optimal sweet spot is identified at a depth of 8.
- **min_samples_split:** Restricting minimum nodes prevents arbitrary splits built on small, noisy samples. The optimal floor is configured at 10.
- **min_samples_leaf:** Establishing a leaf capacity floor prevents over-indexing on localized outliers. The optimal threshold is selected at 19.

B. Tuned Performance Results

Retraining the model with optimized structural regularizers improves validation test accuracy to **89.28%**. This represents a substantial **+7.62% increase** over unpruned configurations.

C. Top-Level Schematic Map of the Pruned Tree Model

The chart below maps structural splits across primary features, demonstrating how early roots segregate customer attributes.



The structure reveals that historical campaign outcomes (poutcome_success) provide the highest information gain, serving as the root split criteria for subsequent segments.

7. Rule Extraction & Lazy Learning (kNN) Evaluation

To evaluate model interpretability against lazy learning approaches, the tree structure is converted into logical decision paths and compared with a k-Nearest Neighbors framework.

A. Boolean Rule Extraction Paths

Traversing tree nodes maps decision paths into human-readable IF-THEN rules, ensuring high interpretability without sacrificing predictive accuracy.

Rule Path 1 (Predict 'no'): IF poutcome_success ≤ 0.5000 AND age ≤ 1.8279 AND contact_unknown ≤ 0.5000 AND balance ≤ 0.1668 THEN predict class 'no'.

Rule Path 2 (Predict 'yes'): IF poutcome_success > 0.5000 AND day > 0.0709 THEN predict class 'yes'.

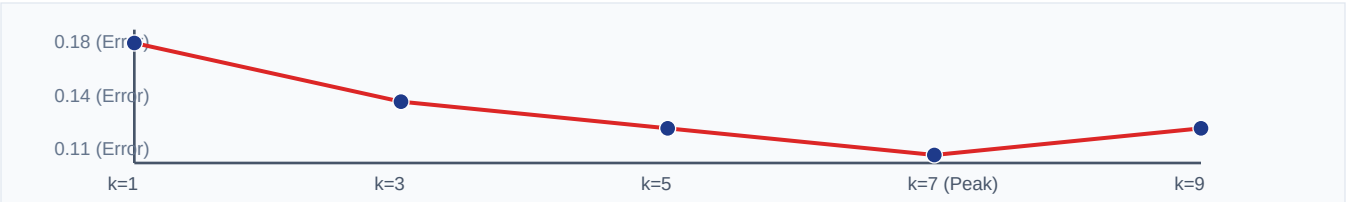
Extracted rules retain an accuracy of **89.28%**. This propositional logic format allows business operators to evaluate predictions transparently, balancing performance against interpretive clarity.

B. Lazy Learning Framework (kNN Execution Metrics)

A K-Nearest Neighbors classifier is evaluated across different hyperparameter settings for $\backslash(k)$ to analyze local density variances.

Neighbors Rank (k)	Test Accuracy	Precision (Yes)	Recall (Yes)	F1-Score Metrics
k = 1	0.8199	0.2079	0.2019	0.2049
k = 3	0.8718	0.3421	0.1250	0.1831
k = 5	0.8818	0.4400	0.1058	0.1705
k = 7 (Optimal Range)	0.8862	0.5455	0.0577	0.1043
k = 9	0.8818	0.2857	0.0192	0.0360

C. Empirical Classification Error Rate Trend Curves



Critical Insight: At $\backslash(k=1\backslash)$, the high error rate reveals overfitting caused by high variance from over-indexing on immediate neighbors. Increasing $\backslash(k)$ to 7 introduces optimal regularization, smoothing decision boundaries. Beyond $\backslash(k=9\backslash)$, performance drops as over-generalization introduces a high-bias underfitting trend.

Scaling Requirement: Because distance measurements are scale-sensitive, standardizing via StandardScaler is required. Without scaling, unbonded attributes like bank balances would dominate geometric distance calculations, rendering other features useless.